

Assignment 4

Facade Pattern

Topic: Option A — Media/File Conversion Facade

SE-2524

Ulzhan Musakhan

Astana 2026

Environment: Java JDK 17

1. Introduction and Design Overview

My project demonstrates the Facade design pattern using a simulated media conversion system.

The conversion process involves several operations: reading file metadata, selecting a codec, calculating the target bitrate, and processing audio channels.

Without a Facade, the client would need to coordinate these operations directly.

The MediaConversionFacade provides one simplified method, convert(fileName, targetFormat), which handles the sequence of operations.

Pattern roles:

Client: Main

Facade: MediaConversionFacade

Subsystems: MetadataReader, CodecFactory, BitrateReader, AudioMixer

Data object: MediaInfo

UML Class Diagram

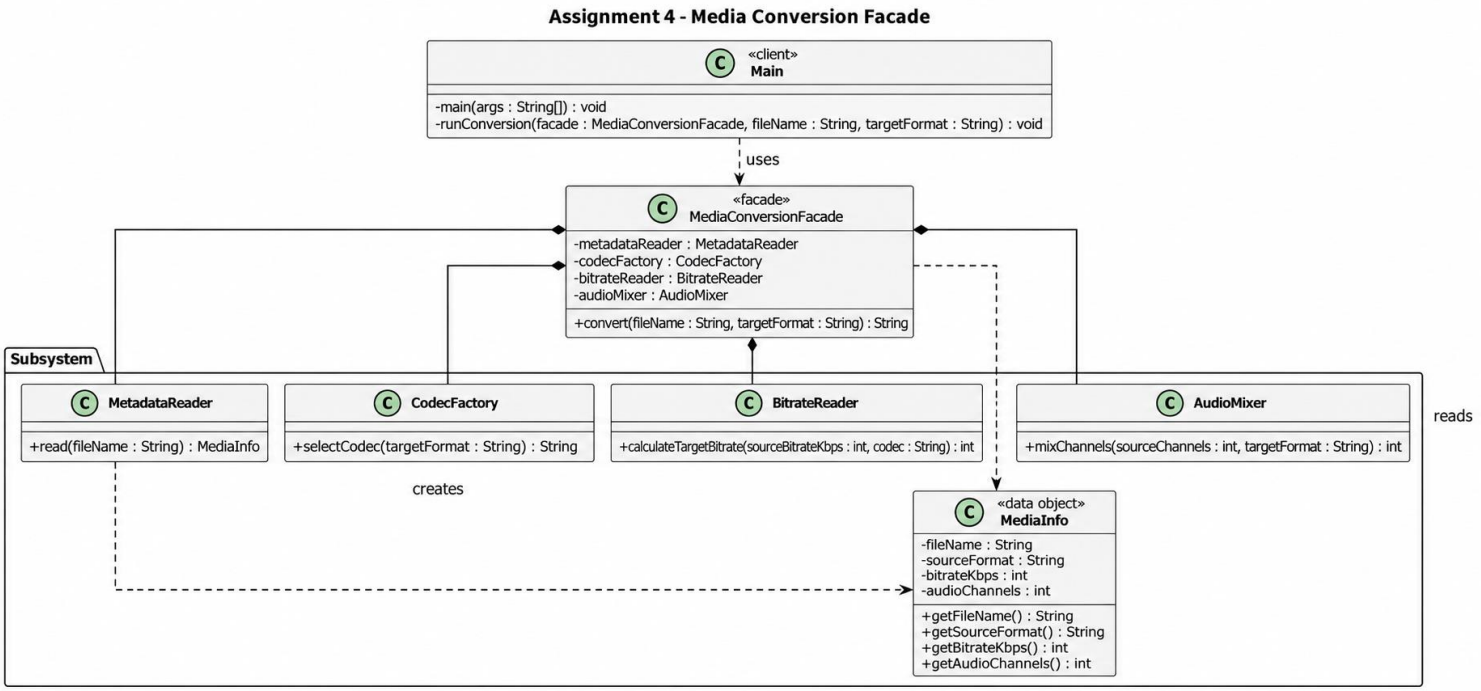


Figure 1. Media Conversion Facade class structure.

The diagram shows that Main depends on MediaConversionFacade. The Facade coordinates four subsystem objects. MetadataReader creates MediaInfo, which contains the data used during conversion.

2. Clean Code Evidence

2.1 Meaningful Names

```java
public String selectCodec(String targetFormat) {  1 usage  httpskwikxs

    if (targetFormat == null || targetFormat.isBlank()) {
        throw new IllegalArgumentException(
                "Target format cannot be empty"
        );
    }

    String format = targetFormat.trim()
            .toLowerCase(Locale.ROOT);

    return switch (format) {
        case "mp4" -> "H264";
        case "mkv" -> "H265";
        case "mov" -> "PRORES";
        case "avi" -> "MPEG4";
        default -> throw new IllegalArgumentException(
                "Unsupported target format: " + format
        );
    };
}
```

Benefit: In CodecFactory.java, the method selectCodec() clearly describes its purpose, while targetFormat identifies the input parameter. These meaningful names make the codec selection logic easier to understand without additional comments.

2.2 Single Responsibility Principle

```
20          MediaInfo media = metadataReader.read(fileName);
21
22          String codec = codecFactory.selectCodec(targetFormat);
23
24          String format = targetFormat.trim()
25                  .toLowerCase(Locale.ROOT);
26
27          int targetBitrate = bitrateReader.calculateTargetBitrate(
28                  media.getBitrateKbps(),
29                  codec
30          );
31
32          int targetChannels = audioMixer.mixChannels(
33                  media.getAudioChannels(),
34                  format
35          );
```

Benefit: In MediaConversionFacade.java, the convert() method coordinates four subsystem operations: metadataReader.read(), codecFactory.selectCodec(), bitrateReader.calculateTargetBitrate(), and audioMixer.mixChannels(). Each subsystem handles a specific responsibility, while the Facade only coordinates their execution.

2.3 Information Hiding

```
5       private final MetadataReader metadataReader =  1 usage
6               new MetadataReader();
7
8       private final CodecFactory codecFactory =  1 usage
9               new CodecFactory();
10
11      private final BitrateReader bitrateReader =  1 usage
12              new BitrateReader();
13
14      private final AudioMixer audioMixer =  1 usage
15              new AudioMixer();
16
17      public String convert(String fileName, String targetFormat) {  1 usage
18
```

Benefit: In MediaConversionFacade.java, the subsystem objects are stored in private final fields. The client accesses the conversion process through the public convert() method without depending directly on the subsystem classes. This reduces coupling and keeps the client code simple.

2.4 Small and Focused Methods



```
5       public int mixChannels(int sourceChannels, String targetFormat) {  1 usage  httpskwikxsi
6
7           if (sourceChannels <= 0) {
8               throw new IllegalArgumentException(
9                       "Audio channels must be positive"
10              );
11          }
12
13          if (targetFormat == null || targetFormat.isBlank()) {
14              throw new IllegalArgumentException(
15                      "Target format cannot be empty"
16              );
17          }
18
19          String format = targetFormat.trim()
20                  .toLowerCase(Locale.ROOT);
21
22          int maxChannels = switch (format) {
23              case "mp4", "avi" -> 2;
24              case "mkv", "mov" -> 6;
25              default -> throw new IllegalArgumentException(
26                      "Unsupported audio format: " + format
27              );
28          };
29
30          return Math.min(sourceChannels, maxChannels);
31      }
32  }
```

Benefit: In AudioMixer.java, the mixChannels() method focuses on calculating the output number of audio channels. It validates input values, determines the maximum supported channels, and returns the result using Math.min(). This makes the logic easier to understand, test, and maintain.

2.5 Error Handling

```
44          } catch (IllegalArgumentException e) {
45              throw new IllegalArgumentException(
46                      "Conversion failed: " + e.getMessage(), e
47              );
48          }
```

Benefit: In MediaConversionFacade.java, the catch (IllegalArgumentException e) block adds a clear "Conversion failed" message and preserves the original exception as its cause. In Main.java, the error is caught and displayed without stopping the remaining demonstration scenarios.

3. Conclusion

The Media Conversion Facade simplifies a multi-step conversion process behind one public method.

The Facade hides subsystem coordination, codec selection, bitrate calculation, and audio-channel processing from the client. However, the subsystem classes remain independently accessible to advanced clients.

This implementation demonstrates how Facade reduces coupling between the client and the subsystem while preserving separation of responsibilities.

4. GitHub Repository

https://github.com/httpskwiksx/facade_pattern

5. References

Course Lecture 6 - Facade Design Pattern.

Assignment 4 - Facade Pattern Instructions.

Freeman, E., & Robson, E. Head First Design Patterns.

Refactoring.Guru - Facade Design Pattern.